Optimization - 2.0.18

EE25BTECH11018 - Darisy Sreetej

March 26, 2026

Question

For the data given below, how many machines can be assigned to an operator to minimize idle time of the operator and machines ?

Activity	Time (minutes)
machine loading + unloading	2
machining	4
walking from one machine to the next	1

1. 1
2. 2
3. 3
4. 4

(EC 2007)

Solution - Identifying Components

Given:

1. Service time (loading + unloading):

$$S = 2 \text{ min} \tag{1}$$

2. Machining time:

$$M = 4 \text{ min} \tag{2}$$

3. Walking time:

$$W = 1 \text{ min} \tag{3}$$

Solution - Operator Cycle time

Consider **n** machines :

$$\begin{aligned}\text{Service time} &= nS = 2n \\ \text{Walking time} &= (n-1)W \\ &= (n-1)\cdot 1 \\ &= n-1\end{aligned}$$

The **operator cycle time** i.e, the total time taken by the operator to complete one full round of all machines and come back to the starting point

$$\begin{aligned}T_o &= nS + (n-1)W \\ &= 2n + (n-1) \\ &= 3n - 1\end{aligned}$$

Solution - Applying Max-Min conditions

Operator should not be idle:

$$T_o \geq M \tag{4}$$

Substituting:

$$\begin{aligned} 3n - 1 &\geq 4 \\ 3n &\geq 5 \\ n &\geq \frac{5}{3} = 1.67 \end{aligned}$$

Thus ,

$$n \geq 1.67 \tag{5}$$

Solution - Applying Max-Min conditions

Machine should not wait too long:
The operator should return before the machine experiences excessive idle time.

$$T_o \leq M + S \tag{6}$$

(The machine runs automatically for time M .After completion, it can tolerate at most one service time delay S .Beyond this, the machine remains idle excessively . So , its $M + S$)

$$\begin{aligned} 3n - 1 &\leq 4 + 2 \\ 3n &\leq 7 \\ n &\leq \frac{7}{3} = 2.33 \end{aligned}$$

Thus ,

$$n \leq 2.33 \tag{7}$$

Solution - Combining Bounds

from (5) , (7) :

$$1.67 \leq n \leq 2.33 \tag{8}$$

Since number of machines must be an integer $n \in \mathbb{Z}$
Hence, the optimal number of machines assigned to the operator is 2 (option 2)

Python Code

```python
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt

# defining Parameters
t_lu = 2 # loading + unloading
t_m = 4 # machining time
t_w = 1 # walking time

n = cp.Variable(integer=True)

# operator cycle time
T_o = n * t_lu + (n - 1) * t_w

idle_time_mismatch_expr = cp.abs(T_o - t_m)
objective = cp.Minimize(idle_time_mismatch_expr)

constraints = [n >= 1]
```

Python Code

```
problem = cp.Problem(objective, constraints)
problem.solve()

optimal_n = int(np.round(n.value))
minimized_idle_time = round(problem.value, 2)

print(f"Optimal Number of Machines (n): {optimal_n}")
print(f"Minimized Idle Time : {minimized_idle_time} minutes")

n_values = np.arange(1, 6)

T_o_values = n_values * t_lu + (n_values - 1) * t_w

idle_time_mismatch = np.abs(T_o_values - t_m)

plt.figure(figsize=(8, 5))
plt.plot(n_values, T_o_values, marker='o', label='Operator Cycle
    Time (To)')
```

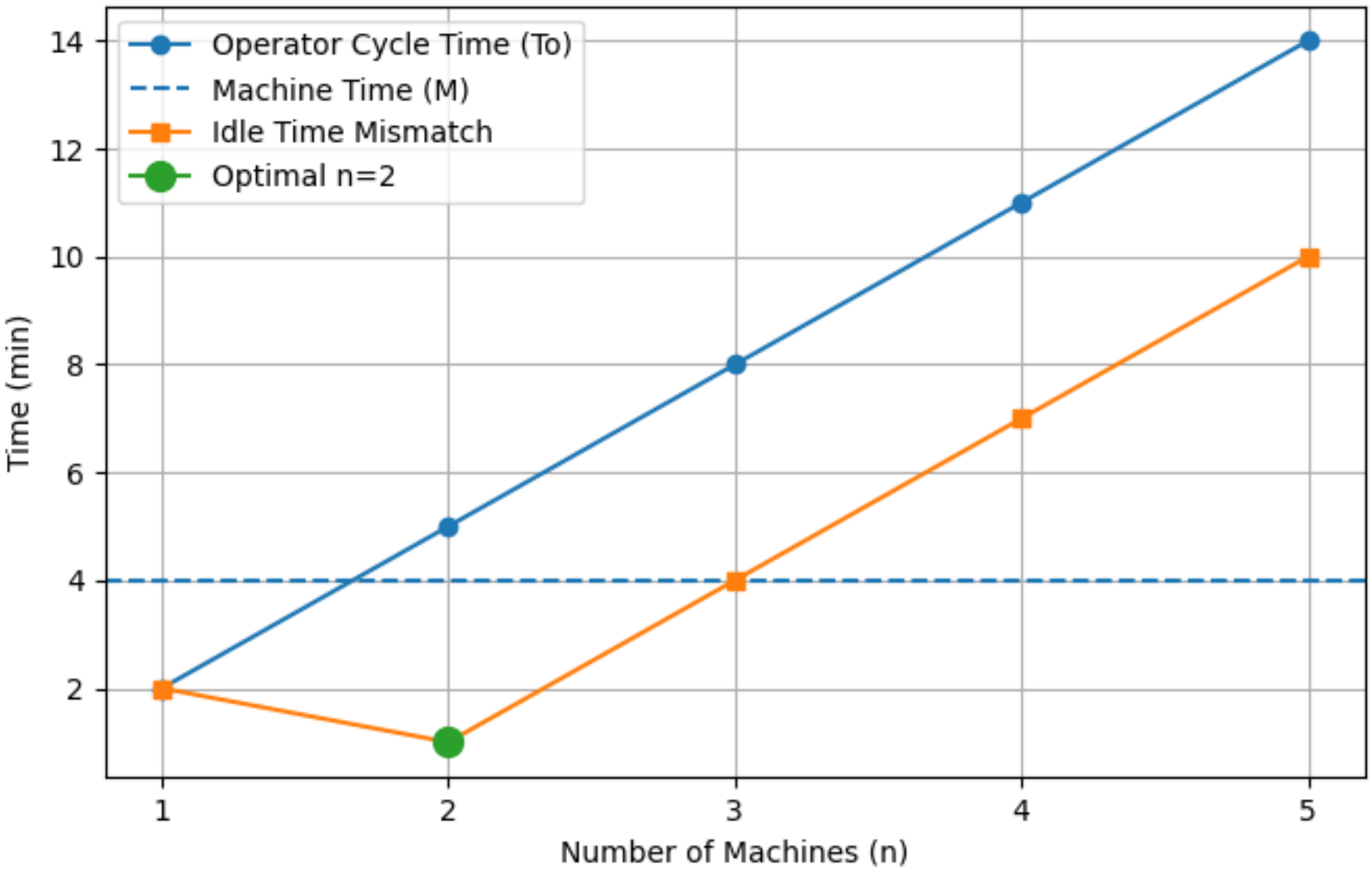
Python Code

```
plt.axhline(y=t_m, linestyle='--', label='Machine Time (M)')
plt.plot(n_values, idle_time_mismatch, marker='s', label='Idle
    Time Mismatch')

plt.plot(optimal_n, minimized_idle_time, marker='o', markersize
    =10, label=f'Optimal n={optimal_n}')

plt.xlabel('Number of Machines (n)')
plt.ylabel('Time (min)')
plt.title('Optimal Machine Assignment')
plt.xticks(n_values)
plt.grid(True)
plt.legend()
plt.show()
```

Plot


Operator Cycle Time (To)
Machine Time (M)
Idle Time Mismatch
Optimal n=2
Time (min)
Number of Machines (n)
14
12
10
8
6
4
2
1
2
3
4
5